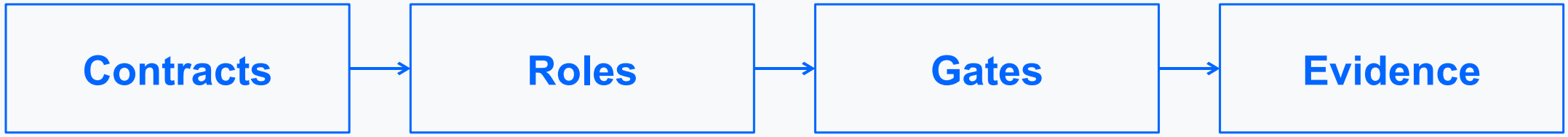
Managing Multi-AI Teams:

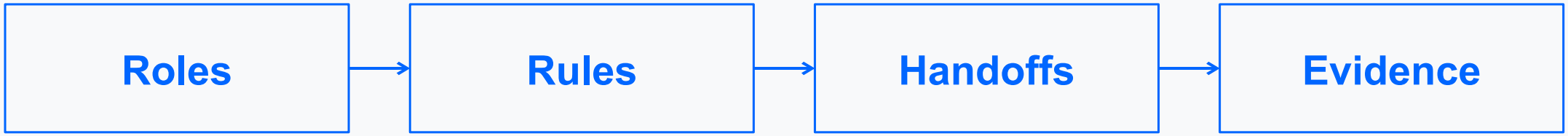
Orchestration to Production

Core stack: Python 3.12 · Pydantic · CrewAI · JupyterLab



Local fixtures · No LLM or reasoning-model calls · API cost $0.00

Architecture & Interaction Patterns



- Roles assign work; rules gate actions; topology routes evidence.
- Compare independent, peer, centralized, and hybrid paths in Lab 2.
- Bound attempts; stop or repair using explicit failure evidence.

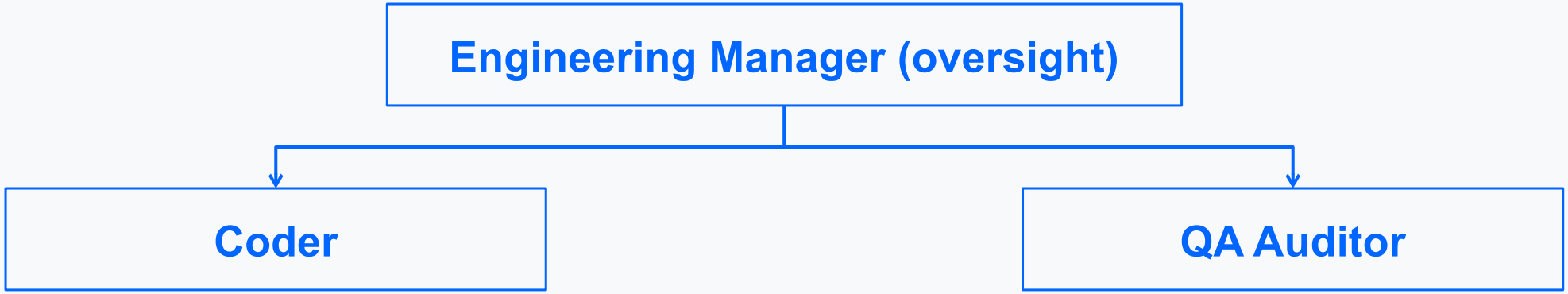
Agent-to-Agent (A2A) Contract Handoffs




```
DevDeliverable:
    code_snippet: str
    unit_tests_passed: bool
    dev_team_signature: str
```

- Reject malformed output and failed QA.
- Scoped memory; typed local API.
- Return SecurityAuditResult.

Hands-On Lab: Code Review Pipeline



- Remove Manager writing access and add architect_task to Coder context.
- Audit the typed patch; admit valid output to qa_inbox.
- Optional bridge: create a Crew, run kickoff, display Markdown.

Hands-On Lab & Homework Deliverable



- Portfolio: three repaired labs + Capstone + ADR with interaction map.
- Factory / CI/CD: candidate revision + checks + gate decisions.
- Prove malfunction, conflicting evidence, stale state, and bounded repair.

Environment & Execution Logistics

- Local: Python 3.12 + Git + JupyterLab at repository root.
- Git sync + dependency install before class; cloud setup is not supplied.
- Evaluation: 3 labs × 20 + Capstone 20 + ADR/defense 20 = 100.

Offline after setup · Existing dependencies · Assessment unchanged